

SSH Honeypot & SOC Monitoring Platform

Author: V. Gedeon

Project: Cowrie-based SSH Honeypot & SOC

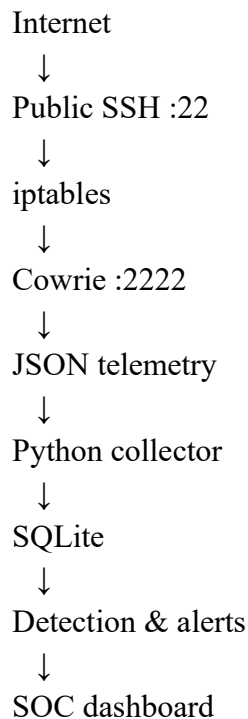
Repository: github.com/vgedeon01/honeypot-project

1. Executive Summary

This project is a defensive security monitoring platform built around an intentionally exposed SSH honeypot.

I deployed **Cowrie** on an Ubuntu VPS, redirected public SSH traffic from port 22 into the honeypot, collected the resulting telemetry, stored it in SQLite, classified attacker commands, generated alerts, and visualized the results through a local SOC dashboard.

The final system provides an end-to-end pipeline:



The project was tested manually and then exposed to unsolicited Internet traffic. The honeypot subsequently observed automated SSH authentication attempts, credential testing, reconnaissance, and post-authentication system fingerprinting.

The project demonstrates practical experience with:

- Linux administration
- SSH security
- Firewall Configuration
- Network configuration
- Honeypot deployment
- Python development
- Log processing
- SQLite
- Detection engineering
- SOC monitoring
- Incident investigation
- Systemd services
- Git/GitHub

2. Project Objectives

The project was built to answer a simple question:

Can I build and operate a small SOC pipeline that detects and analyzes real SSH attacks?

The main objectives were:

1. Deploy an SSH honeypot safely.
2. Keep the real VPS administration interface separated from the honeypot.
3. Collect structured security telemetry.
4. Detect suspicious commands automatically.
5. Store events in a searchable database.
6. Build a simple SOC dashboard.
7. Validate the detections with controlled attacks.
8. Analyze unsolicited attacks from the public Internet.

The focus was on building a working system rather than simply installing an existing honeypot.

3. Security Background

SSH is one of the most frequently exposed administrative services on Internet-facing Linux systems. Public SSH services are regularly scanned and targeted by automated systems attempting common usernames, passwords, and post-authentication commands.

A honeypot provides a controlled environment where this behaviour can be observed without giving attackers access to the real operating system.

Cowrie was selected because it provides an interactive SSH environment and produces structured telemetry about connections, authentication attempts, commands, and sessions.

The project adds a custom monitoring layer around Cowrie to turn this raw telemetry into security events and alerts.

4. System Architecture

The system consists of five main components.

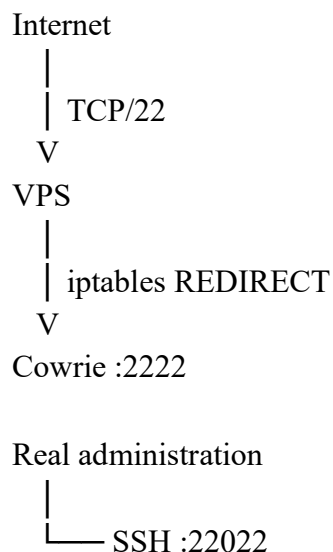
Cowrie

Cowrie provides the SSH honeypot and emulated Linux shell.

Network layer

Public TCP/22 is redirected to Cowrie on TCP/2222.

The real SSH service operates separately on TCP/22022.



Collector

collector.py monitors Cowrie's JSON log and converts events into structured database records.

Database

SQLite stores:

events
sessions
commands
downloads
alerts

SOC dashboard

A Flask dashboard displays live statistics including:

- sessions
- commands
- alerts
- alert severity
- command categories
- source IPs
- top commands

The dashboard listens only on:

127.0.0.1:5000

It is therefore not exposed to the Internet.

5. Network & Security Configuration

Security separation was an important part of the deployment.

Service	Port	Purpose
Public SSH	22	Cowrie honeypot
Cowrie	2222	Internal honeypot listener
Real SSH	22022	Administration
Dashboard	5000	Local monitoring

The public SSH port is redirected with iptables, while the real SSH service was moved away from port 22 and configured for key-based authentication.

The administrative port is restricted through the VPS firewall.

The dashboard is bound to localhost and therefore cannot be accessed directly by external attackers.

This gives the system a clear separation:

Attack traffic → Cowrie

Administrator → Real SSH

SOC monitoring → Local dashboard

6. Telemetry Collection

Cowrie writes events to:

`/opt/cowrie/var/log/cowrie/cowrie.json`

The custom collector continuously processes new events.

It records:

- connection information
- source IP
- session ID
- authentication attempts
- usernames/passwords
- commands
- session duration
- downloads
- alerts

The collector also performs event deduplication so that repeated processing does not create duplicate records.

Commands are classified into behavioural categories:

Category	Examples
Recon	whoami, id, uname, ls

Download	wget, curl
Execution	bash, python, chmod
Persistence	crontab, authorized_keys
Credential access	/etc/passwd, /etc/shadow
Networking	nc, ping, dig
Destructive	rm, mkfs, dd

The database then becomes the central source for both analysis and visualization.

7. Detection Engineering

The detection engine uses transparent command-based rules.

Severity is assigned according to behaviour:

Severity	Example
Low	Basic reconnaissance
Medium	Network discovery
High	Credential access, downloads, persistence, destructive commands

For example:

```
whoami
↓
recon
↓
low alert
```

while:

```
cat /etc/shadow
↓
credential_access
↓
high alert
```

The detection rules were tested independently before deployment.

Detection test result: 21/21 tests passed.

This provided a basic regression test for the command classifier.

The system intentionally focuses on **observable behaviour**, rather than attempting to identify the person behind an IP address.

8. Deployment & Validation

The project was deployed as three systemd services:

```
cowrie.service  
cowrie-soc-collector.service  
cowrie-soc-dashboard.service
```

All three components were tested independently and then together.

Controlled test

An external test connection was made to the public port 22.

Commands included:

```
whoami  
ls  
cat /etc/shadow  
exit
```

The expected results were observed.

whoami produced a reconnaissance alert.

cat /etc/shadow produced a high-severity credential-access alert.

This verified the complete pipeline:

SSH connection



Cowrie



JSON log



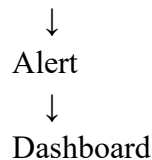
Collector



SQLite



Detection



The test confirmed that the public attack path was reaching Cowrie rather than the real SSH service.

9. Real-World Attack Results

After deployment, the honeypot began receiving unsolicited SSH activity from the public Internet. The anonymized investigation dataset contains **135 unique source addresses**, with activity ranging from simple connection attempts to multi-stage post-authentication behaviour.

The investigation was performed using the collected session, authentication, command, alert, client fingerprint, and file-download telemetry.

The main behaviours observed were:

- automated credential testing
- repeated successful honeypot authentication
- host fingerprinting
- system and process enumeration
- SSH persistence attempts
- credential modification
- cron inspection
- destructive cleanup
- payload/file staging

The important distinction is that these are **observed behaviours inside the honeypot**. A successful Cowrie login does not represent compromise of the underlying VPS.

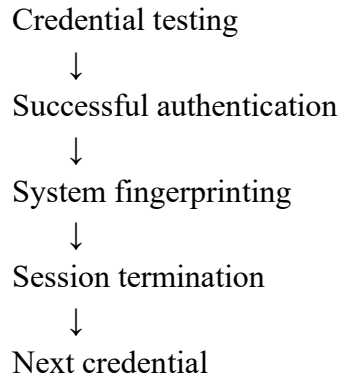
9.1 From Scanning to Post-Authentication Activity

Most observed activity was automated and relatively shallow.

One campaign generated hundreds of short sessions. After successful authentication, the client repeatedly executed a single system-fingerprinting command:

```
uname -s -v -n -r -m
```

The same pattern occurred across many sessions, indicating a workflow of:



This type of activity is useful as a baseline because it represents common automated SSH behaviour.

However, several other campaigns progressed considerably further.

9.2 The Deepest Observed Campaign

The most significant investigation involved **Attacker 102** in the anonymized dataset.

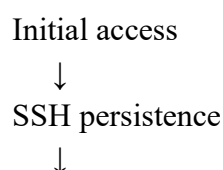
The source generated:

- **68 events**
- **3 sessions**
- **18 commands**
- **8 alerts**
- activity over approximately two minutes

The first successful session lasted approximately 60 seconds and began with a root login.

The attacker immediately attempted to establish SSH persistence by manipulating `.ssh/authorized_keys`. This was followed by host reconnaissance, password modification, process/system inspection, cron inspection, and destructive cleanup.

The sequence was significantly more involved than the automated fingerprinting campaigns:



Credential modification
↓
Host reconnaissance
↓
Process / resource discovery
↓
Cron inspection
↓
Cleanup / destructive activity

The detection engine identified:

- **2 persistence behaviours**
- **1 destructive behaviour**
- multiple reconnaissance behaviours

The investigation therefore moved from simple “SSH brute force” into a genuine **post-authentication attack workflow**.

9.3 Repeated Persistence Campaign

The investigation also revealed that the persistence behaviour was not isolated.

Several sources used the same distinctive SSH client fingerprint and executed substantially identical commands to modify `.ssh/authorized_keys`.

The repeated workflow was:

Successful root login
↓
Modify SSH configuration/files
↓
Replace `authorized_keys`
↓
Install attacker-controlled public key
↓
Attempt to preserve future access

The same public-key persistence pattern appeared across multiple sources and sessions.

From a SOC perspective, this is significant because it provides a possible **campaign-level correlation**.

The individual source addresses are different, but the combination of:

- SSH client fingerprint
- command sequence
- persistence technique
- attacker-controlled key material
- timing and session behaviour

creates a stronger relationship than IP address alone.

This is the type of correlation that could be developed into a future detection rule.

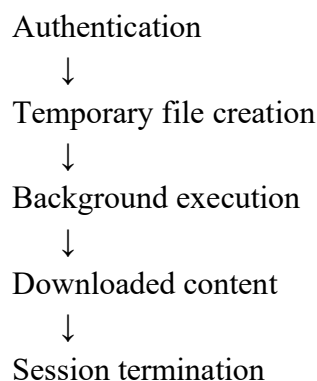
9.4 Payload Staging

A separate source showed another interesting behaviour.

After one failed authentication attempt, the same client successfully authenticated and immediately executed a command that created a temporary file in /tmp and placed the process into the background.

Cowrie then recorded a file-download event and stored the resulting content with a SHA-256 hash.

The sequence was approximately:



The extremely short session — less than one second after authentication — suggests the interaction was automated rather than manually operated.

This is a particularly useful example because it shows why **file-download telemetry matters**. A command that appears relatively simple becomes much more significant when correlated with the subsequent file event.

9.5 Overall Assessment

The observed Internet activity can therefore be divided into several behavioural levels:

Level 1 — Scanning

Connection / authentication attempts

Level 2 — Fingerprinting

uname / hostname / system identification

Level 3 — Enumeration

Processes / memory / filesystem / environment

Level 4 — Persistence

SSH key modification / cron inspection

Level 5 — Payload activity

File staging / downloads / cleanup

The majority of observed sessions remained at the first two levels. A smaller number progressed into deeper post-authentication behaviour.

That difference is one of the most valuable findings from the deployment: **not every SSH attack behaves the same way, and session-level telemetry allows the analyst to distinguish automated scanning from more involved attack workflows.**

10. SOC Investigation Case Studies

10.1 Case Study 1 — Deep Post-Authentication Attack

Case 1 — Privileged Access and Persistence Attempt

Severity: High | **Confidence:** High

On 27 September 2026, Attacker 102 successfully authenticated to the honeypot using a root account. The session then attempted to modify `authorized_keys`, change the root password, enumerate system resources and inspect cron and running processes. Cleanup commands were also observed.

The strongest indicator was the attempted installation of an attacker-controlled SSH key. This is consistent with an attempt to maintain access independently of the original credentials.

Evidence: successful root authentication, `authorized_keys` modification, credential-change command, CPU/memory/process/cron enumeration, cleanup activity.

Detection: persistence and destructive-behaviour rules generated high-severity alerts.

Assessment: High-confidence evidence of post-authentication persistence activity. The ultimate purpose of maintaining access cannot be determined from the honeypot telemetry alone.

10.2 Case Study 2 — Repeated SSH Persistence Campaign

Case 2 — Repeated SSH Persistence Pattern

Severity: High | **Confidence:** High**

Multiple anonymized sources performed highly similar SSH persistence actions, including modification of `.ssh/authorized_keys`. Several sessions shared client-fingerprint characteristics and the same command structure.

The repeated pattern suggests common automation or tooling. It is not sufficient to attribute the activity to a specific actor or campaign.

Detection opportunity: correlate successful authentication, `authorized_keys` modification, client fingerprint and recurrence across sources.

10.3 Case Study 3 — Rapid Payload Staging

Case 3 — Short-Lived Payload Staging

Severity: Medium–High | **Confidence:** High**

One source successfully authenticated and, within approximately one second, wrote data to `/tmp`, launched a background process and generated a Cowrie file-download event before disconnecting.

The purpose of the downloaded file was not established. The behaviour is consistent with automated payload staging.

Detection gap: the current command classifier did not generate an alert for this sequence, despite the subsequent download telemetry. This identifies an area for improving detection coverage.

10.4 Case Study 4 — Automated System Fingerprinting

Case 4 — Automated System Fingerprinting

Severity: Low | **Confidence:** High

A separate source generated a large number of short SSH sessions. After successful authentication, the same system-identification command was executed repeatedly before the sessions ended.

The observed command collected basic operating-system and kernel information. No persistence, credential modification, destructive commands, or payload activity were observed in these sessions.

This behaviour is consistent with **automated system fingerprinting**: determining what type of system has been reached before deciding whether further activity is worthwhile.

The case is useful because it provides a contrast with the more advanced sessions in the dataset. A successful SSH login alone does not indicate that an attacker has attempted to establish persistence or perform destructive activity. The subsequent behaviour is what determines the investigative significance of the session.

10.5 Investigation Summary

The investigations showed several different levels of activity rather than one continuous attack sequence.

The dataset included:

- automated SSH credential testing;
- repeated system fingerprinting;
- successful post-authentication reconnaissance;
- attempts to establish SSH persistence;
- credential modification;
- file-download and payload-staging activity;
- cleanup and destructive commands.

The most significant investigations were the sessions that progressed beyond authentication into **persistence, credential modification, payload activity, or destructive behaviour**.

A key finding from the project was that these behaviours could be distinguished by correlating multiple telemetry sources:

Authentication → Session → Commands → Client fingerprint → File activity → Alerts

This provided more context than looking at individual SSH log entries in isolation.

The investigation also exposed limitations in the current detection logic. In particular, some short-lived payload-staging activity generated useful Cowrie telemetry without triggering an alert. This provides a concrete area for improving the detection rules.

Overall, the project demonstrated a complete defensive workflow:

Collect → Detect → Correlate → Investigate → Identify gaps → Improve detection

Rather than treating every successful SSH connection as equally important, the investigation focused on the behaviour that followed authentication and the evidence supporting each assessment.

11. Future Improvements

The current system provides a strong foundation for further development.

Potential improvements include:

- behavioural sequence detection
- command obfuscation detection
- automated IOC extraction
- richer dashboard visualizations
- session timelines
- malware/download analysis
- longer-term attack statistics
- automated detection testing
- stronger outbound network restrictions
- integration with external threat-intelligence sources

A future detection could identify sequences such as:

Login
↓
Recon
↓
Download
↓
Execute
↓
Persistence

rather than evaluating every command independently.

12. Conclusion

This project demonstrates an end-to-end defensive security workflow.

I designed and deployed an SSH honeypot, separated it from the real administration interface, built a custom telemetry collector, created a SQLite security database, implemented command detection, generated alerts, and developed a local SOC dashboard.

The system was validated through controlled attacks and then exposed to real Internet traffic. The collected data demonstrated automated credential testing and post-authentication reconnaissance against the public SSH service.

The project combines practical experience in:

Linux • SSH • Networking • Python • SQLite • Detection Engineering • SOC Monitoring • Honeypots • Incident Analysis • Git

The main outcome is not simply a working honeypot, but a complete pipeline for turning unsolicited network activity into structured security data that can be investigated and presented.